2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops

MARITACA: from textual use case descriptions to behavior models

Leydi Erazo, Eliane Martins
University of Campinas – UNICAMP
Institute of Computing
Campinas – Brazil
leydi.paruma@students.ic.unicamp.br,
eliane@ic.unicamp.br

Juliana Galvani Greghi
Computer Science Department
University of Lavras
Lavras – Brazil
juliana@dcc.ufla.br

***Abstract*— It is well known in Software Engineering that the cost of eliminating requirements faults increases when these faults are revealed. Therefore, it is important to express requirements in a way that it would be possible to validate them as early as possible in the development cycle. Use Cases (UC) are a popular format to represent requirements. On the other hand, states machines are a notation amenable for verification, either using for example, model-checking or model animation, and also are a popular notation for model-based test case derivation. However, building a state machine from pure textual requirements is not trivial for most practitioners, in particular, a model with properties that allow them to be handled by a tool. UCs are described in pure natural language, in general, and the manual extraction of state machine models from them can be time-consuming and error-prone. In this work, we present an approach that helps the extraction of state machine models from UC descriptions. These descriptions are in the form of a semi-structured language, instead of pure textual form. The goal is to provide practitioners with a preliminary version of a state model that can be further refined to become input for tools like test case generators. As a proof of concept, a prototyping tool, MARITACA, was developed that uses Natural Language Processing techniques to extract the state machines. The text also shows a validation of the model obtained from the tool, to demonstrate the applicability of the approach.**

Keywords— software product line; use case specification; state machine;

I. INTRODUCTION

As computer systems become more present in our everyday life, failures are more visible to the users, and their impact costs also increase. A study about bug characteristics on three large, representative open source software has shown that semantic bugs, i.e., inconsistencies with the requirements, are the major root cause of failures (about 80%) [1]**.** This same study also shows that semantic bugs increase as the software evolves. Part of this problem of non-conformance with requirements can be attributed to the fact that requirements are generally written in natural language, as they must be understandable by non-technical stakeholders. However, the use of natural language to express system requirements can lead to misunderstandings (due to ambiguities), as well as incompleteness and inconsistency (due to difficulties in discovering relationships among requirements). The problem is that these bugs remain undetected until later phases of software life-cycle when the cost of fix them are higher.

The use of more specialized notations, like use cases, are also often used, but UML gives no specific format to write them, which ends up with a pure natural language text. Ideally, the use of models with a more precise syntax and semantics would be helpful, such as state models. The advantage of such model are: i) the represent the system usage scenarios in a more precise way than in a textual description of use case flows; ii) they can be graphically represented, which makes them easier to analyze than a natural language text; iii) they are amenable to analysis by tools (model animation, model checking) and iv) they can be used for test case generation.

The difficulty is that, in practice, the creation of state models from use case descriptions requires a solid modeling expertise, which are still rare among practitioners. Therefore, having a way to automatically transform a natural language specification into a state model is helpful, as evidenced by several works in this area [2][3][4][5]. In this study, we propose the use of Natural Language Processing (NLP) techniques to extract a state model from use cases described in a structured natural language format. In this way, a preliminary model is presented to the practitioner, so that (s)he can complete and refine it to be processable by tools, in particular, test case generation tools, our first concern. What we expect is that the fact of transforming the preliminary model into a test model can help practitioners discover omissions and inconsistencies, in this way, contributing to requirements improvements.

The paper describes MARITACA (state MAchine geneRatIon from TextuAl use CAses), a proof-of-concept tool which extracts state machine models from use cases. To show evidence of its applicability, we present its use in a case study. Maritaca is intended to generate models for single systems, but also for a family of products. Product family (PF) refers to a set of similar software products that share a common set of features. Since there is a high number of potential products in a PF, product testing automation is essential.

The remainder of the paper is organized as follows. Section 2 presents the related works on extraction of Analysis models

2325-6664/17 $31.00 © 2017 IEEE
DOI 10.1109/DSN-W.2017.33

from textual requirements. The approach proposed to write use case descriptions is presented in Section 3. Behavior model extraction is presented in Section 4. In section 5 we show the feasibility of the proposed approach, and observe its advantages and limitations. Finally, Section 6 presents the conclusions and future works.

II. RELATED WORK

Approaches for automatic extraction of Analysis models from use cases descriptions have been proposed in various works. The AGADUC (Automated Generation of Activity Diagrams from UCs) [6] process, as it name indicates, proposes a systematic way to generate UML Activity Diagrams from textual descriptions of use case flows. The tool AREUCD provides an automated support to this process. The process requires use cases to be described in a structured way, but with a simple syntax that is facilitates the use by practitioners. Then, this description is transformed to a more formal notation from which the Activity Diagram is extracted. For each UC, AGADUC generates an activity diagram for the basic and alternative flows, and separate diagrams for each subflow or extension points. Our proposed approach, on the other hand, generates an state machine model to represent the flow between UCs.

Another proposal is RUCM (Restricted Use Case Modeling) [7], a structured notation for use cases specification. RUCM is used by the aToucan tool to extract various Analysis models such as class**,** activity and sequence models [8], [9]. Rules and algorithms for automatic extraction of state machines for RUCM descriptions were also proposed [3]. Our approach is based on their approach, in that we extended RUCm to cope with exception handling, as well as with variability in UC descriptions. The rules to generate state machine models from UCs were also extended. RUCM was intended for single product, whereas we generate models for a product family. Somé et al. also proposed a template for UC description composed of a static part, describing the system states (preconditions and postconditions, among others) and a dynamic part describing the use case flows . As in RUCM, there are also restrictions to the natural language description to allow the state model extraction. The algorithm generates state transitions models from use cases by looking at a use case as a network of connected scenarios. A tool, UCed [10] supports Somé's approach. We get inspiration from these approaches, which were focused on single systems, to propose our approach for PF modelling and testing. In particular, our tool was inspired by our previous work in that domain, which extracted a state model from a pure textual specification [11]. In this sense, this work is close to the approach presented in [4], which model PF requirements using an enhanced use case description. The approach describes functional variation points at use case level, to allow the automatic generation of the behavior of a specific chosen product. The model represents control flow among use cases based on the contract, as in our approach. However, they use a more restricted language to specify the use cases than we do.

III. USE CASE SPECIFICATION (UCS)

Use cases are widely used to describe mainly functional requirements (FR), but also some non-functional requirements (NFR), like fault-tolerance [12]**.** Use cases describe an external view of the system, i.e., they show how the system interacts with its actors, which can represent people, external systems or hardware devices. The UCS depicts use cases, actors, communication associations between actors and use cases, use case relationships, whereas the Use Case Specifications are usually expressed as pure textual language. UML provides no specific notation for specifying UCS, so it is common practice to adopt some template to help reading and reviewing use cases. We adopted the template defined by [7]**,** Restricted Use Case Modeling (RUCM). We adopted this template because i) it gets inspiration from various existing templates and, most important in our case, ii) it uses a structured language, in that it proposes a set of restriction rules to write UCS in order to facilitate automated UC analysis necessary to extract the behavior models. However, differently from RUCM, we adopt a scenario style for describing a use case, in that a scenario represents a particular path through a use case from the point of view of a primary actor. A scenario is described by a list of steps, in which each step is a declarative statement with no branching [13]. The only exception are alternative scenarios, which are initiated by an IF statement that gives the condition for this scenario to be executed. Another difference we have with respect to RUCM template is to allow the separate specification of exceptional behavior, necessary to specify dependable systems.

Finally, our UCS approach is aimed to allow the specification of Product Families (PF). In this context, it is necessary to consider common, variable and product-specific requirements for the whole set of products in a family. Therefore, we also included variability information in the RUCM description, as it was originally proposed for a single product. Two types of variability are considered [14]: (i) *use case variability*, in which the whole use case is a variant (indicated by <<optional>>), and (ii) *scenario variability*, in which the variants are alternative scenarios in a use case. In this case, there is a variation point in the use case, represented as an alternative variation scenario; a tag indicates the feature related to the variation point. In a previous work [15] we presented the UCS template, but a brief overview is given here, for the paper to be self-contained.

Each Use Case (UC) has a unique identifier, a name, a description, Pre- and Post-conditions, primary and secondary actors, dependency and generalization, as in other templates. The Use Case Type and Associated Feature fields are intended to handle variability when specifying a PF. The Use Case Type specifies whether the use case is *mandatory, optional*, or *alternative*. A *mandatory* use case is present in all members of the PF. *Optional* use cases are needed by only some members of the PF, and *alternative* use cases have different versions that are required by different members of the PF [16]**.** Variability can also be specified inside a use case, using variation points, which are places that identify locations at which variation occurs. For a more complete description, please refer to [15]**.** An example of a Use Case Specification is given in Section 5.

IV. BEHAVIOR MODEL EXTRACTION

The same way as in [3], [17], our approach aims to generate a state-machine model representing sequential dependencies among UCs. In general, these dependencies are not explicitly identified in textual descriptions. They can be made explicit through the use of pre- and post-conditions. Precondition is the set of conditions that must be true for the use case to start. Post-condition is the set of conditions that must be true when the use case finishes.

The extracted model is an extended finite state machine, which can be defined as M = < S, initial, start, I, O, V, T >. **S** is a finite set of states, with initial representing the **initial** pseudostate of a UML state machine, and **start** indicating the initial state. Except for the initial pseudo-state, all the others are represented by the pre- and post- conditions of the use cases (UC). **I** is a finite set of input events, which are represented by use cases, as those generally are caused by a request from an actor. **O** represents a finite set of output actions or the outputs generated from the system to an actor. **V** represents a finite set of context variables used in state invariants or in guards. **T** represents a finite set of transitions. Since states represent pre- and post- conditions, and there are as many post-conditions as flows in a use case, there are as many transitions as flows for a given use case (UC). In other words, each transition corresponds to a (basic or other) flow in the UCS, with the use case name as the trigger. The guards differentiate the transitions from the origin to a destination state for a given UC. Guards correspond to the condition step that precedes each non-basic flow. In case of exception flows, this condition is specified with the **VALIDATE THAT** keyword. The **ABORT** keyword, on the other hand, indicates that the system execution can no longer continue. **RESUME STEP** keyword terminates Recoverable flows. This indicates that the exceptional flow goes back to the reference flow, meaning that exception handling succeeded well. **POSTCONDITION** keyword specifies that each flow (basic, alternative, exceptional and variation) should have its own postcondition. **IGNORE** keyword is used to represent a loop, the system does not execute any action when this keyword is present.

The approach also considers *extend* and *include* relationships among use cases. In the extend relationship, an extending UC adds an optional behavior to a base UC. In this sense, it is considered as an alternative flow, and as such, the condition of the extend relationship becomes the guard, whereas the extension UC becomes an output action in the state machine transition. In the include relationship, the behavior of an including UC is inserted into the behavior of a base UC. In this case, the guards obtained from the base UC are combined with the ones of the including UC, according to the flow in which the inclusion occurs.

To ease the automatic extraction of state machine elements presented above, users should adopt some conventions in use case writing. Convention number one, explained before, is that a UCS contains scenarios that represent particular paths through a use case from the point of view of a primary actor. **IF** commands are only allowed in the beginning of alternative or exceptional scenarios. Convention two is that actions performed by the system must be explicitly stated as "The system does something", so that the output actions of a transition are identified. Convention three is that the user should identify the initial state, by indicating the precondition that characterizes this state. In this way, it is possible to connect the **start** state to the indicated state. Convention four is relative to variability representation, in which features are written inside curly brackets "**{}**"in variation flows.

TABLE I. SUMMARY OF EXTRACTION RULES

Rule #	Description
1	Generate an instance of *StateMachine* for the use case model.
2	Generate the initial state for the state machine.
3	Generate an instance of **State**, named as **'start'**, representing the start state of the state machine.
4	Generate an instance of **Transition**. Its trigger is named as **'construct'**. This transition connects the initial state to the start state.
5	Invoke rules 5.1-5.4 to process each use case of the use case model.
5.1	Generate an instance of **State** for each sentence of the precondition of the use case, as long as such a state has not been generated, which is possible because two use cases might have the same preconditions.
5.2	Generate an instance of **State** for each sentence of the postcondition of the basic flow of the use case, as long as such a state has not been generated.
5.3	Connect the states corresponding to the precondition to the states representing the postcondition of the basic flow of the use case with the transition whose trigger is the name of use case.
5.4	Process the postcondition of each alternative flow of the use case.
5.4.1	Generate an instance of **State** for each sentence of the postcondition of each alternative flow.
5.4.2	Connect the states corresponding to the precondition of the basic flow to the states corresponding to the postconditions of the alternative flows with the transition whose trigger is the name of use case.
6	Connect the precondition of the including use case to its postcondition through a transition. Its trigger is the including use case. Its effect is the included use case, and the guard is the conjunction of all the conditions of the condition sentences of the previous steps of the step containing the **INCLUDE USE CASE** sentence.
7	Process the postcondition of each variation flow of the use case.
7.1	Generate an instance of **State** for each sentence of the postcondition of each variation flow. For this rule are used the curly brackets "{}", to determinate if a feature of the PF is selected.
7.2	Connect the states corresponding to the precondition of the basic flow to the states corresponding to the postconditions of the variation flows with the transition whose trigger is the name of use case.
8	Process the postcondition of each exceptional flow of the use case.
8.1	Generate an instance of **State** for each sentence of the postcondition of each exceptional flow.
8.2	Connect the states corresponding to the precondition of the basic, alternative or variation flow to the states corresponding to the postconditions of the exceptional flows with the transition whose trigger is the name of use case. For this rule is used the **VALIDATE THAT** keyword, to determinate if an exception is identified.

As a proof of concept, we implemented our approach in a prototype tool, MARITACA (state MAchine geneRatIon from TextuAl use CAses). Based on our previous experience with TXT2SMM prototype [11], which extracts state machine models from a narrative text, MARITACA applies techniques

from Natural Language Processing (NLP), more specifically, a Part-of-Speech tagger, which processes the words in a text and attaches to each a tag according to its grammatical category (noun, adjectives, verbs and so on). By analyzing the tagged text, the tool uses a set of rules (TABLE I) to categorize the sequence of words that characterize states, transitions, triggers and so on. We used all the rules proposed by [3]**;** we also add 2 rules to handle the exceptional flows and the variability in PFs.

The Traceability Matrix (**TM**) and the Product Map (**PM**) [18] are also inputs to MARITACA, together with the UCS. The PM describes all the features that are required to build desired products from the PF, and is represented as a matrix Features x Products. TM relates features to use cases, and the tool uses this information to show which features are available for a given product. Then the tool selects the use cases to analyze based on this information.

Fig. 1 below shows a schematic view of the extraction steps as explained in the paragraphs above.

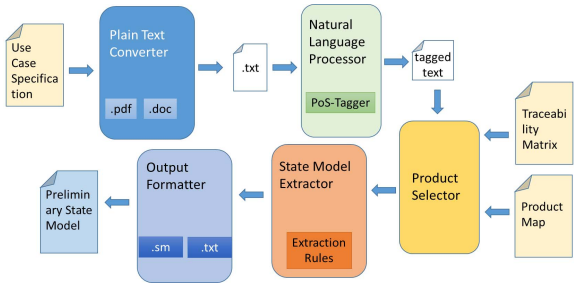

Fig. 1. A schematic view of the model extraction steps.

Next we present an example used to validate the approach.

V. Validation of the Approach

This initial validation of approach had the following objectives: **O1**. to show the feasibility of extracting state machine models from the restricted UCS; **O2**. to validate the extracted model by comparing it to the one manually produced, according to the same rules as presented in TABLE I and **O3**. to observe the weak points of the approach that serve as indications for future research directions. The first objective was achieved with the construction of the MARITACA tool, which served as a proof of concept that the proposed UCS is amenable to automatic extraction of a state model.

For the objectives **O2**, we used as example a Microwave oven system, a product line presented in [19]. The microwave oven system has input buttons for selecting Cooking Time, Start and Cancel, as well as a numeric keypad. It also has a display to show the cooking time left and a microwave heating element for cooking food. Cooking is possible only when the door is closed, when there is something in the oven and when the cooking time is not zero. Additionally, it has the beeper to indicate when cooking is finished, light which is switched on when the door is open and during the cooking. It also is switched off when the door is closed and when cooking stops. The turntable rotates for during cooking and stops when cooking stops.

There is a choice of language for displaying messages. The default language is English, but other possible languages are French, Spanish, German and Italian. Prompts and warning messages are displayed in the display unit. Basic oven has a one-line display; more-advanced ovens can have multi-line displays. The time-of-day option can be used only when the multi-line display is present. The top-of-the-line oven has a recipe cooking feature, which needs an analog weight sensor, a multi-line display and a multi-level power features.

The validation was carried out according to the following steps. First, we specified the system according to the restrictions and conventions established by our approach. Then the specification was input to MARITACA, which produced a preliminary state model. In parallel, a state model was manually built. Both the extracted and the manually produced model were then compared. These steps are presented in the sequel. The section ends with a discussion of the results and the threats to validity.

A. *Modelling the case study*

Although inspired in the specification described by [19]**,** we need to rewrite the use cases to document them according to the template and conventions established in our approach. Therefore, the number of use cases is not the same. Fig. 2 shows the UC diagram for the microwave oven product family.

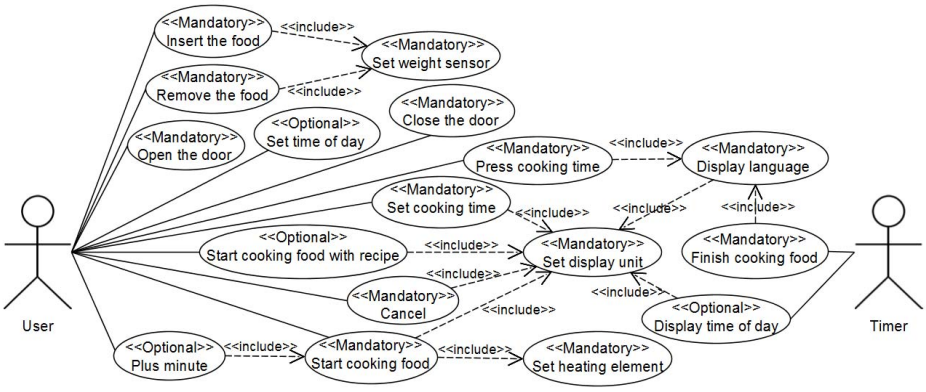

Fig. 2. Use cases diagram for the Microwave oven product line

TABLE II shows an example of use case specified according to the restrictions and conventions presented in Section 3. It is the *Finish cooking food*, a mandatory UC, which means that it is present in all products. For the sake of space, we present a partial view of the specification, omitting fields left blank in the description.

TABLE II. PARTIAL UCS OF THE FINISH COOKING FOOD

Use Case Id: UC012
Use Case Name: Finish cooking food.
Use Case Type: Mandatory.
Cardinality feature: 1..1
Brief Description: The system finishes cooking the food.
Associate feature: Microwave Oven Kernel, Light, Turntable, Beeper.
Primary Actor: Timer.
Secondary Actor: None.
Pre-condition: The door is closed AND the food is inserted AND the time is not zero AND the start button is pressed.
Dependency: INCLUDE USE CASE Display language.
Generalization: None.
Basic Flow Steps (BFS):
1. The timer sends finished message to the system.
2. The system stops cooking.
3. The system sets the time TO zero
4. The system sets the start button TO not pressed.
5. **INCLUDE USE CASE** Display language.
6. The system displays end message.
7. The system clears the display.

Postcondition: The door is closed AND the food is inserted AND the time is zero AND the start button is not pressed.
...
Specific Variation Flow (SVF-1):
1. {the light is selected}
2. The system sets the lamp TO off.

Postcondition: The lamp is switched off.
Specific Variation Flow (SVF-2):
1. {the turntable is selected}
2. The system sets the rotation TO off.

Postcondition: The rotation is off.
Specific Variation Flow (SVF-3):
1. {the beeper is selected}
2. The system sets beep TO on.

Postcondition: The beep is on.
...

This UC illustrates some of the key elements our specification approach. The Basic flow includes the *Display Language* UC, to allow the end message to be displayed in a specific language. Besides, this UC presents three variation points, represented in specific variation flows. The condition for a variation flow to be taken is given as a tag with the corresponding feature (in curly brackets).

B. State machine generation

State models are generated per product, so, it is necessary for the user to select which product (s)he wants. With the information stored on the Traceability Matrix and the Product Map, it is possible for the tool to determine which use cases and which features to select to extract the model.

MARITACA can produce the output in textual form or using the ".sm" notation, which is the input to the State Machine Compiler [20]. In SMC's notation, constraints in guard conditions are represented as logic expressions used in programming languages. Although UML recommends OCL (Object Constraint Language) [21] for constraints specification, its syntax is not easy to use, therefore, we preferred SMC's notation that is easy to understand and to refine even for a tester not used to write state machines. Other reasons for use this format are: (i) it is possible to compile the model to various languages (Java, C, etc), so that the tester can animate the model to validate whether the scenarios specified in the requirements are being satisfied, and (ii) this format is the input to a test generation tool developed by the group in a previous work. In this paper, for the sake of space, we only show the textual notation. An excerpt of the state machine extracted for one product is depicted in Fig. 3. In the textual format, **Pre** indicates the source state, **Trigger** is the use case name, **Guard** indicates a guard condition and **Effect** the action.

The complete descriptions of all UCs and other artifacts used by MARITACA, as well as the extracted state machine for three products are available at the website [22].

```
Pre: The door is closed AND the food is not inserted AND the time is zero AND the start button is not pressed
Trigger: Open the door
Guard: [the door is closed AND the food is not inserted AND the time is zero AND the start button is not pressed] /
Effect: The system sets the door TO open
Post: The door is open AND the food is not inserted AND the time is zero AND the start button is not pressed ._.
...
Pre: The door is open AND the food is not inserted AND the time is zero AND the start button is not pressed
Trigger: Insert the food
Guard: [ The door is open AND the food is not inserted AND the time is zero AND the start button is not pressed ] /
Effect: The system sets weight sensor TO on
Post: The door is open AND the food is inserted AND the time is zero AND the start button is not pressed  AND The
weight sensor is on
...
Pre: The door is open AND the food is not inserted AND the time is zero AND the start button is not pressed
Trigger: Close the door
Guard: [the door is open AND the food is not inserted AND the time is zero AND the start button is not pressed] /
Effect: The system sets the door TO closed ._.
Post: The door is closed AND the food is not inserted AND the time is zero AND the start button is not pressed ._.
...
```

Fig. 3. Excerpt of a state machine description generated by MARITACA

C. Results

With regards to objective **O2** we compare the extracted model with the one manually generated from the UCS. Ideally, we would compare the extracted model with the one in the textbook from which we extract the example system. However, as we did change the UCS to adequate to our template, it was no longer possible to compare with the state models in the textbook. We manually produced a model generated according to the rules, to serve as the expected result to compare with the one produced by MARITACA. The objective **O3** is a discussion about the results that is presented in subsection D.

The Meld tool [23] was used for the comparison, and an excerpt of this is depicted in Fig. 4. Some minor differences were found (like the use of comma in the manual model, instead of "AND", or the use of "The system" starting every action taken by the system)..

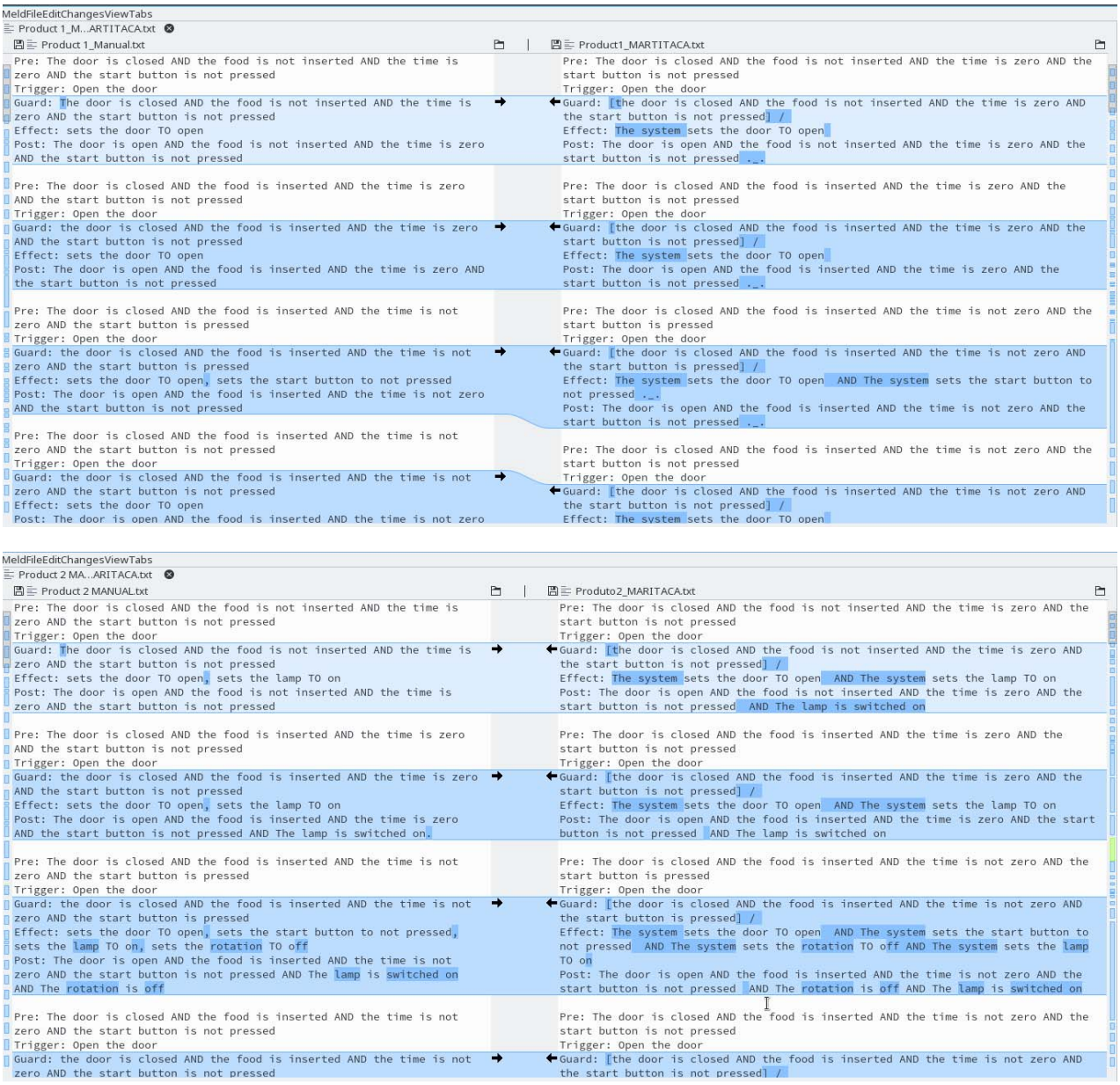

Fig. 4. Comparison between manual generated and extracted models.

We also verified whether the state machines were syntactically and semantically correct. To make it semantically correct, it was necessary to refine the model, to make it connected and eliminate redundant states. It is worth noting that with the conventions we adopted (see Section IV), redundancy can be quite reduced. Duplicate states can occur, for example, when UC1 contains: "**Pre-condition**: The door is closed AND the food is inserted AND the time is not zero AND the start button is pressed" and in another point of the same or other UC the user writes: "**Pre-condition**: The door is closed AND the food is ***in the oven*** AND the time is not zero AND the start button is pressed"

D. Threats to validity

The primary goal of this study was to validate the restricted UCS and the approach for deriving the state model from UCS. The development of the MARITACA tool served as a proof of concept of the feasibility of automating the extraction of the state model from a use case specification. A first threat to validity is the fact that the example application is not a real-world application. Although the example was from a textbook, the system is quite complex, and we were able to generate the models for different products. Besides, this example fits to our goal that was mainly to validate the proposed UCS and the MARITACA tool.

A main threat to the external validity concerns the restrictions imposed in writing the use cases. This writing is based on RUCM, whose authors performed a controlled experiment in an industrial setting. Their results show that the restriction rules are not hard to apply and that there were improvements in the overall system specification [7]. Our approach uses a subset of their keywords, but we added extensions for exception handling and for the specification of product families. All the elements presented in Section 3 and exemplified in Table II must be followed when constructing a use case specification. The point here is that they do introduce restrictions to practitioners used with pure textual languages. It is worth mentioning that the quality of the extracted model highly depends on the way the use cases are written. The more the user sticks to the template and conventions, the better the extracted model, and as a consequence, the easier it is to refine the model. We may also not forget that the restrictions limit the inherent ambiguity of the pure natural language text, which improves the overall quality of the requirements.

In order to address the first threat, there is an ongoing work, in which the approach is being applied to a real-world case study, and the specification is written by third parties

VI. Conclusions and Future Works

This paper presented an approach to the automatic extraction of a state model from the use case specifications. A tool, MARITACA, was developed to support the approach. Our main objective was to help practitioners to construct a test model from which test cases can be generated. Although the notation used to express the state machine model is compatible with a test case generator developed in our group, the approach is "tool agnostic", in the sense that the test model can be used with other tools. So much that in our experiments with the approach, we are also using the Parteg tool [24]. Our approach also aims to help improve requirements, even indirectly: when completing the model, the tester can discover omissions and inconsistencies in the requirements. The tool also contributes to better writing of use cases, as the extracted model is as good as the use cases from which it derives. We also intend to develop a use case editor to guide users in writing their use cases. It is also envisaged as future works to use the approach with third parties, so as to evaluate its usefulness for end users.

Acknowledgment

The authors would like to thank the CAPES for the support. They are also grateful to the European DEVASSES Project.

References

[1] L. Tan, C. Liu, Z. Li, X. Wang, Y. Zhou, and C. Zhai, "Bug characteristics in open source software," *Empir. Softw. Eng.*, vol. 19, no. 6, pp. 1665–1705, 2014.

[2] S. Somé, "Beyond Scenarios: Generating State Models from Use Cases," *ICSE 2002 Work. Scenar. state Mach. Model. algorithms, tools.*, pp. 456–462, 2002.

[3] T. Yue, S. Ali, and L. Briand, "Automated transition from use cases to UML state machines to support state-based testing," *Proc. 7th Eur. Conf. Model. Found. Appl.*, pp. 115–131, 2011.

[4] C. Nebut, Y. Le Traon, and J.-M. Jezequel, "System testing of product lines: From requirements to test cases," in *Software Product Lines*, Springer, 2007, pp. 447–477.

[5] L. Kof, "Translation of Textual Specifications to Automata by Means of Discourse Context Modeling," in *International Working Conference on Requirements Engineering: Foundation for Software Quality*, Springer, 2009, pp. 197–211.

[6] M. El-Attar and J. Miller, "AGADUC : Towards a More Precise Presentation of Functional Requirement in Use Case Models," in *Software Engineering Research, Management and Applications, 2006. Fourth International Conference on*, 2006, pp. 346–353.

[7] T. Yue, L. C. Briand, and Y. Labiche, "A Use Case Modeling Approach to Facilitate the Transition towards Analysis Models :," in *International Conference on Model Driven Engineering Languages and Systems*, Springer Berlin, 2009, pp. 484–498.

[8] T. Yue, L. C. Briand, and Y. Labiche, "aToucan: An automated framework to derive UML analysis models from use case models," *ACM Trans. Softw. Eng. Methodol.*, 2015.

[9] T. Yue, L. C. Briand, and Y. Labiche, "An Automated Approach to Transform Use Cases into Activity Diagrams," *Proc. 6th Eur. Conf. Model. Found. Appl.*, pp. 337–353, 2010.

[10] S. Somé, "UCEd - Use Case Editor," 2007. [Online]. Available: http://www.site.uottawa.ca/~ssome/Use_Case_Editor_UCEd.html. [Accessed: 26-Apr-2017].

[11] J. G. Greghi, E. Martins, and A. M. B. R. Carvalho, "Semi-automatic Generation of Extended Finite State Machines from Natural Language Standard Documents," in *Dependable Systems and Networks Workshops (DSN-W)*, 2015, pp. 45–50.

[12] C. M. F. Rubira, R. de Lemos, G. R. M. Ferreira, and C. F. Filho, "Explicit Representation of Exception Handling in the Development of Dependable Component-Based Systems," *Softw. Pract. Exp.*, pp. 195–236, 2005.

[13] R. Wirfs-Brock and J. Chwartz, "The art of writing use cases," in *Tutorial for OOPSLA Conference*, 2001.

[14] R. P. De Oliveira, "Understanding and guiding software product lines evolution based on requirements engineering activities," Universidade Federal da Bahia, 2015.

[15] L. Erazo, E. Martins, and J. Galvani Greghi, "Modeling Dependable Product-Families : from Use Cases to State Machine Models," in *Latin-American Symposium on Dependable Computing*, 2016, p. 4.

[16] K. Kang, S. Cohen, J. Hess, W. Novak, Peterson, and Spencer, "Feature-Oriented Domain Analysis (FODA) Feasibility Study," 1990.
[17] C. Nebut, F. Fleurey, Y. Le Traon, and J.-M. Jezequel, "A requirement-based approach to test product families," in *Software Product-Family Engineering*, Springer Berlin, 2003, pp. 198–210.
[18] R. P. De Oliveira, E. Insfran, S. Abrahao, J. Gonzalez-Huerta, D. Blanes, S. Cohen, and E. S. De Almeida, "A feature-driven requirements engineering approach for software Product Lines," in *Software Components, Architectures and Reuse (SBCARs), 2013 VII Brazilian Symposium on.*, 2013, pp. 1–10.
[19] H. Gomma, *Designing software product line with UML: From Use Cases to Pattern-Based Software Architectures*. Addison-Wesley, 2004.
[20] C. W. Rapp, "SMC - The State Machine Compiler." [Online]. Available: http://smc.sourceforge.net/. [Accessed: 26-Apr-2017].
[21] The Object Management Group, "Unified Modeling Language: Superstructure Version 2.0," 2004.
[22] L. Erazo, E. Martins, and J. Greghi, "MARITACA," 2017. [Online]. Available: http://www.students.ic.unicamp.br/~ra161251/. [Accessed: 26-Apr-2017].
[23] K. Willadsen, "Meld." [Online]. Available: http://meldmerge.org/. [Accessed: 26-Apr-2017].
[24] S. Weißleder, D. Sokenou, and B. H. Schlingloff, "Reusing State Machines for Automatic Test Generation in Product Lines," *1st Work. Model. Test. Pract. (MoTiP 2008)*, p. 19, 2008.